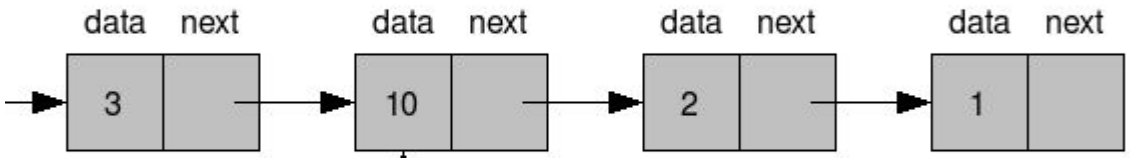
Tabela Hash

Estruturas de Dados

adalberto@computacao.ufcg.edu.br

Estruturas vistas até agora...



Busca em O(n)

É possível melhorar search, add, remove para O(1)?

Estrutura de tabela, mapa, dicionário

Utiliza o conceito de chave → valor

Operações

- put(chave,valor)
- get(chave)
- remove(chave)

Sigla (chave)	Capital (valor)
"MS"	"Campo Grande"
"PB"	"João Pessoa"
"SP"	"São Paulo"
"RS"	"Porto Alegre"

Estrutura de tabela, mapa, dicionário

Utiliza o conceito de chave → valor

Operações

- put(chave,valor) = t[ind_1] = valor
- get(chave) = t[ind_2]
- remove(chave) = t[ind_3] = null

Sigla (chave)	Capital (valor)
"MS"	"Campo Grande"
"PB"	"João Pessoa"
"SP"	"São Paulo"
"RS"	"Porto Alegre"

Estrutura de tabela, mapa, dicionário

Utiliza o conceito de chave → valor

Operações

- put(chave,valor)
- get(chave)
- remove(chave)

Sigla (chave)	Capital (valor)
"MS"	"Campo Grande"
"PB"	"João Pessoa"
"SP"	"São Paulo"
"RS"	"Porto Alegre"

Universo de chaves: U = {"MS", "PB", "PA", "AM", "PE", "RJ", "CE", ….}

Chaves em uso: K = {"MS", "PB", "SP", "RS"}

Estrutura de tabela, mapa, dicionário

Utiliza o conceito de chave → valor

Operações

- put(chave,valor)
- get(chave)
- remove(chave)

Que estrutura pode nos dar acesso direto?

Sigla (chave)	Capital (valor)
"MS"	"Campo Grande"
"PB"	"João Pessoa"
"SP"	"São Paulo"
"RS"	"Porto Alegre"

Universo de chaves: U = {"MS", "PB", "PA", "AM", "PE", "RJ", "CE", ….}

Chaves em uso: K = {"MS", "PB", "SP", "RS"}

Estrutura de tabela, mapa, dicionário

Utiliza o conceito de chave → valor

Operações

- put(chave,valor)
- get(chave)
- remove(chave)

Que estrutura pode nos dar acesso direto?

Sigla (chave)	pos
"MS" →	0
"PB" →	1
"SP" →	2
"RS" →	3

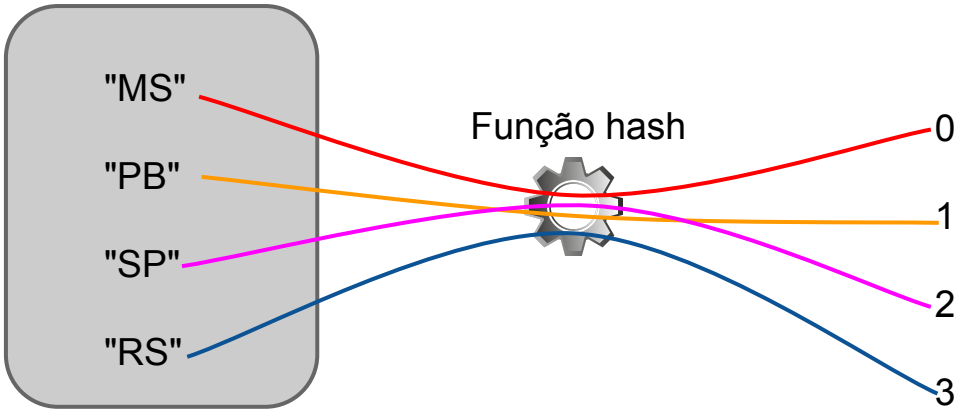
	Capital
0	"Campo Grande"
1	"João Pessoa"
2	"São Paulo"
3	"Porto Alegre"

Universo de chaves: U = {"MS", "PB", "PA", "AM", "PE", "RJ", "CE", ….}

Chaves em uso: K = {"MS", "PB", "SP", "RS"}

Tabela Hash

- Função de Hash: calcula um valor inteiro a partir de uma chave.
  - Determinística



Capital
"Campo Grande"
"João Pessoa"
"São Paulo"
"Porto Alegre"

Tabela de acesso direto

Universo de chaves: U = {"MS", "PB", "PA", "AM", "PE", "RJ", "CE", ….}

Tabela: |U| = |T| = m

hash(K key) → {0,1,2,3,...,m-1}

- Sobrejetora: toda chave é mapeada para um valor
- Injetora: cada valor é referente a apenas uma chave

Tabela de acesso direto

Universo de chaves: U = {"MS", "PB", "PA", "AM", "PE", "RJ", "CE", ….}

Tabela: |U| = |T| = m

hash(K key) → {0,1,2,3,...,m-1}

- Sobrejetora: toda chave é mapeada para um valor
- Injetora: cada valor é referente a apenas uma chave

put(k, e) : T[hash(k)] = e

remove(k): T[hash(k)] = null

search(k): return T[hash(key)]

O(1)

Tabela de acesso direto

Universo de chaves: U = {"MS", "PB", "PA", "AM", "PE", "RJ", "CE", ….}

Tabela: |U| = |T| = m

E se |U| >> |T|?

hash(K key) → {0,1,2,3,...,m-1}

- Sobrejetora: toda chave é mapeada para um valor
- Injetora: cada valor é referente a apenas uma chave

Tabela Hash

Temos mais chaves que posições

Permitir que mais de uma chave possa ser mapeada para um mesmo índice

Função de Hash não é perfeita

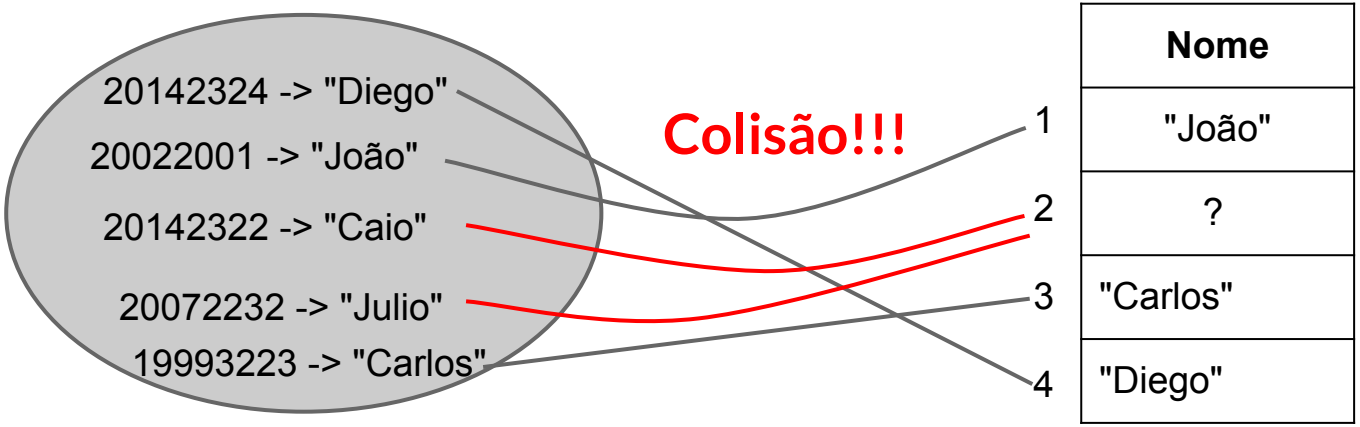


Tabela Hash (como resolver colisões?)

- Endereçamento fechado (chaining)
  - Mais de um elemento no mesmo slot da tabela
- Endereçamento aberto
  - Um elemento por slot → busca por um slot vazio

Tabela Hash (como resolver colisões?)

Endereçamento fechado (chaining)

Mais de um elemento no mesmo slot da tabela

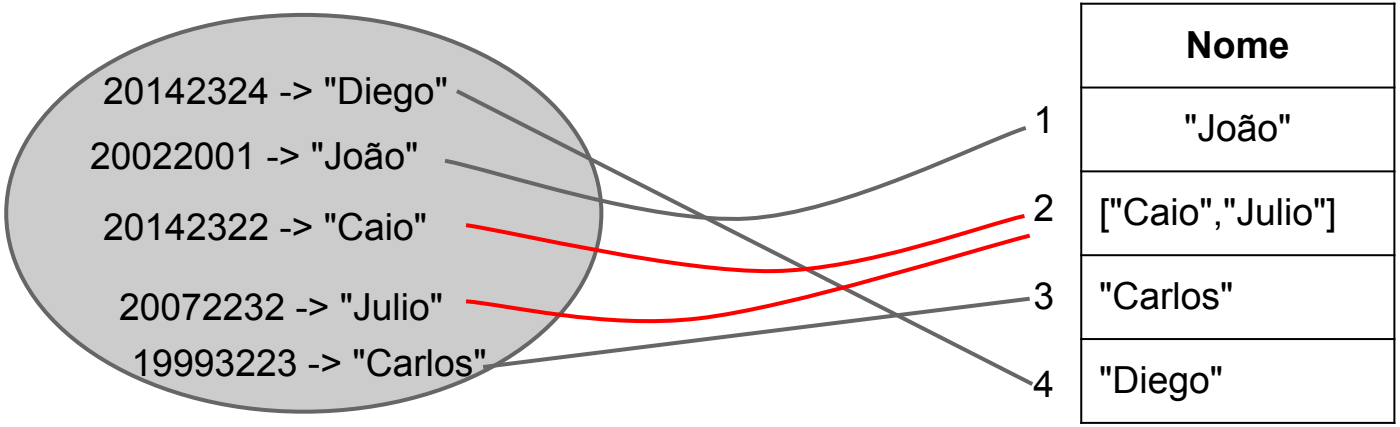
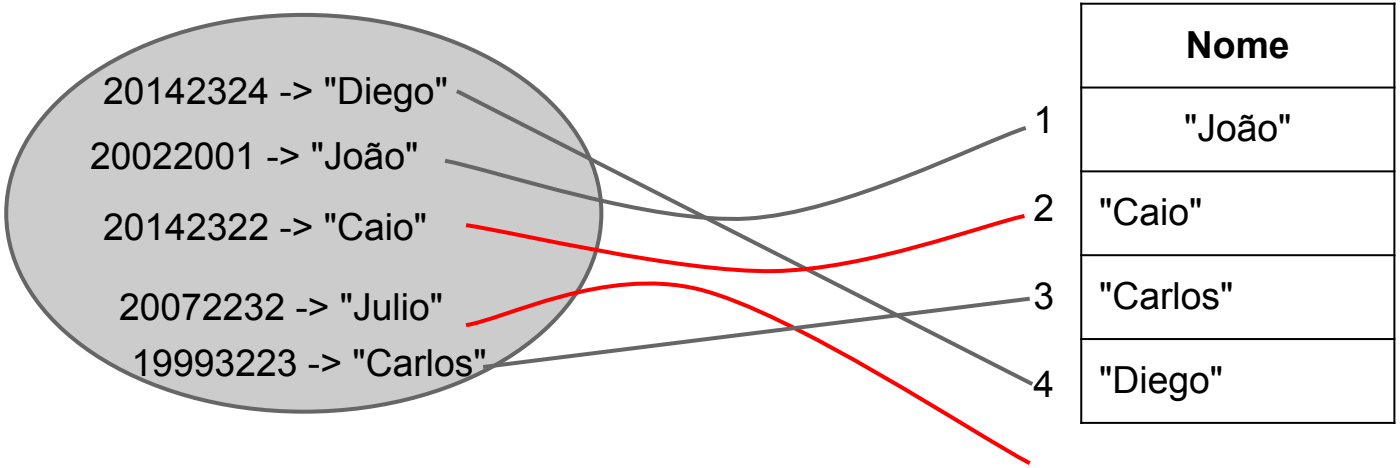


Tabela Hash (como resolver colisões?)

Endereçamento aberto

Um elemento por slot → busca por um slot vazio

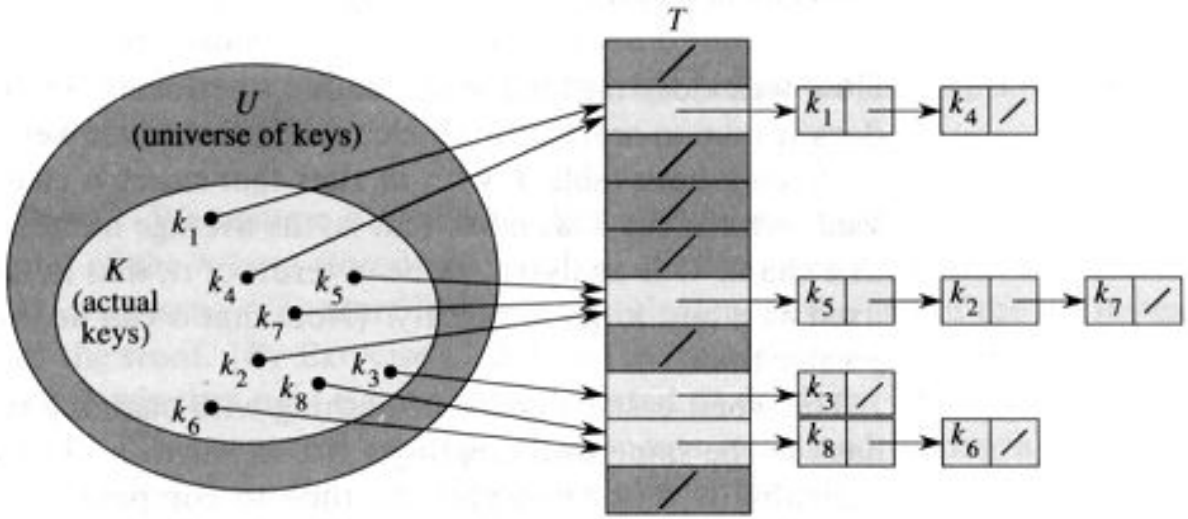


Tabelas Hash
Endereçamento Fechado

Tabela Hash (Endereçamento Fechado)

Simulação

Tabela Hash (Endereçamento Fechado)



Operações:

Inserir(x)

Inserir x na cabeça de T[h(x)];

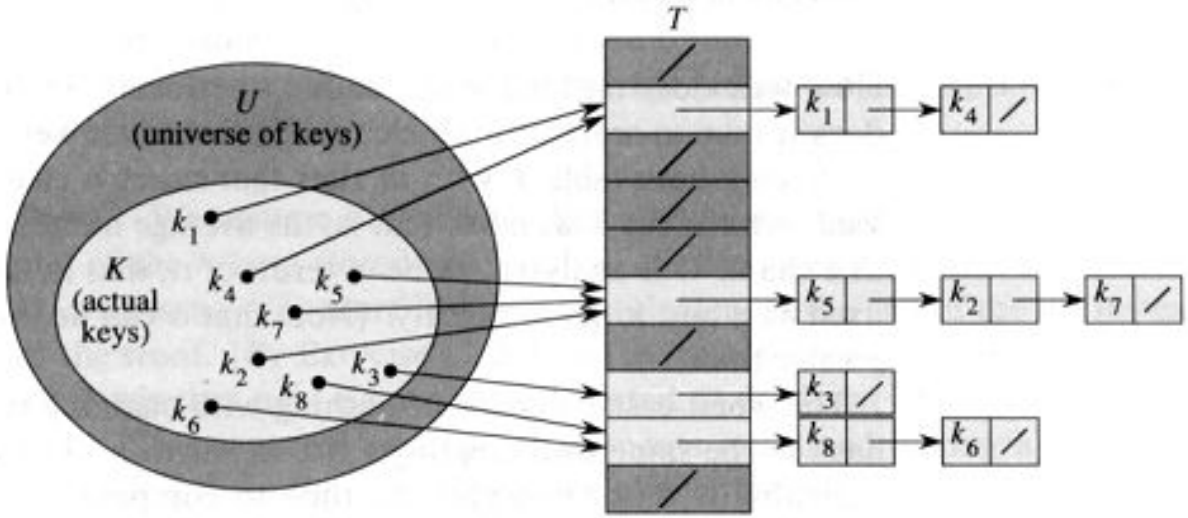
Remover(x)

Deletar x da lista T[h(x)];

Pesquisar(k)

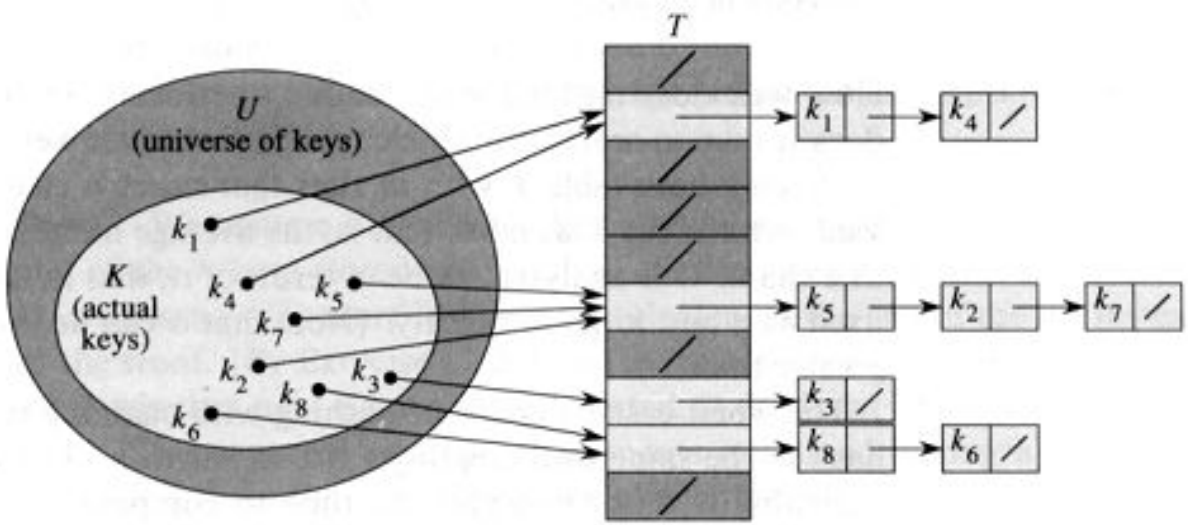
Buscar x na lista T[h(k)];

Tabela Hash (Endereçamento Fechado)



|U| >> |T|

Tabela Hash (Endereçamento Fechado)



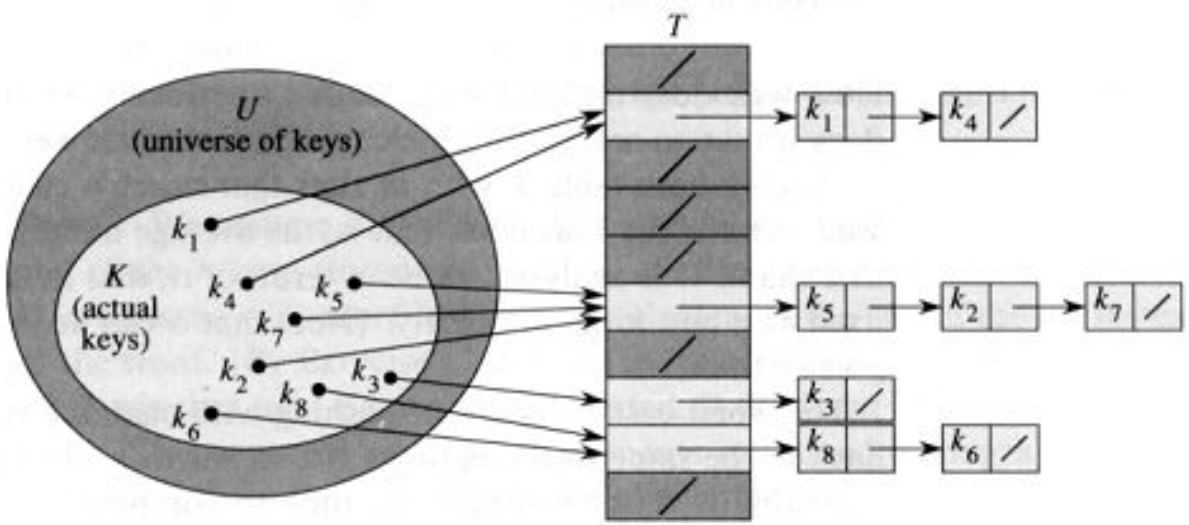
Hashing uniforme:

Cada elemento tem igual probabilidade de cair em qualquer slot!

⇩

Cada slot da tabela possui em média a mesma quantidade de elementos!

Tabela Hash (Endereçamento Fechado)



Hashing uniforme:

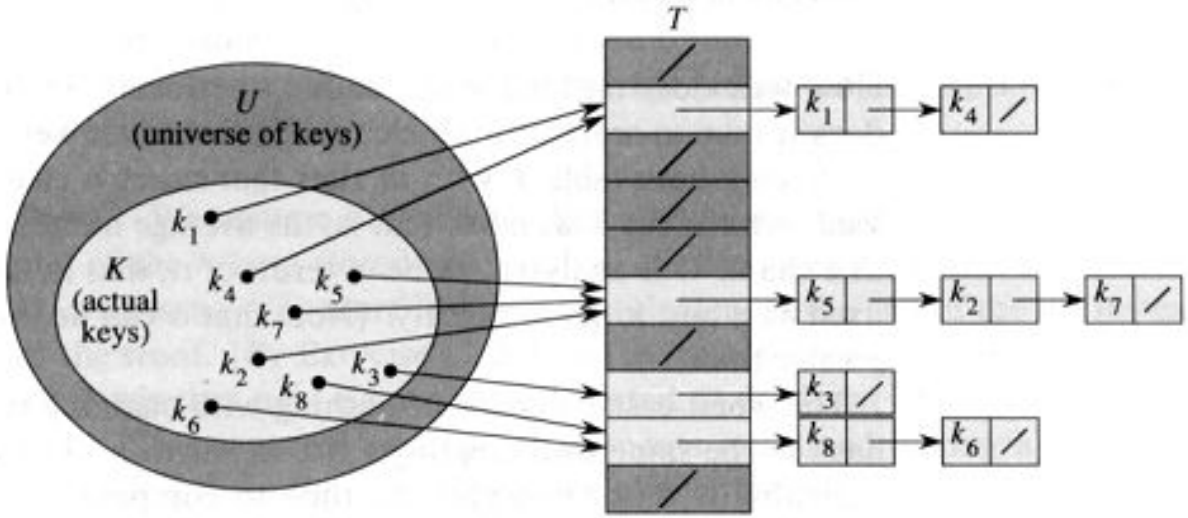
Cada elemento tem igual probabilidade de cair em qualquer slot!

⇩

fator de carga

$|U|/|T| = \alpha$

Tabela Hash (Endereçamento Fechado)



Aplicar a função hash

Operações: $\Theta(1+\alpha)$

Inserir(x)
  Inserir x na cabeça de T[h(x)];
Remover(x)
  Deletar x da lista T[h(x)];
Pesquisar(k)
    Buscar x na lista T[h(k)];

Tabela Hash (Endereçamento Fechado)

Exemplo:

Tabela para armazenar o cpf de vocês (m=10).

h(k) = k % 10

XXXXXXXXX-XY

Como escolher uma boa função de hashing?

Tabela Hash (Endereçamento Fechado)

Métodos das funções de hashing:

- Método da divisão
- Método da multiplicação

Foco:

- Minimizar colisões
- Satisfazer hashing uniforme

Tabela Hash (Endereçamento Fechado)

Método da Divisão

$$f(k) = k \% m$$

Tabela Hash (Endereçamento Fechado)

Método da Divisão

$$f(k) = k \% m$$

Dicas:

- Não escolha um m tal que tenha um divisor d bem pequeno
- Não escolha $m = 2^r$,
- Escolha m sendo um número primo que não seja próximo das potências de 2 ou 10

Tabela Hash (Endereçamento Fechado)

Método da Divisão

- Suponha que um conjunto de 2000 elementos será armazenado em uma tabela hash onde o fator de carga estimado é 3. Qual seria a melhor função de hash para o problema usando o método da divisão?

Tabela Hash (Endereçamento Fechado)

Método da Divisão

- Suponha que um conjunto de 2000 elementos será armazenado em uma tabela hash onde o fator de carga estimado é 3. Qual seria a melhor função de hash para o problema usando o método da divisão?

$h(k) = k \% \mathbf{m}$

2000/3 ~ 667

Primos: 673, 677,683, 691, 701, 709, 719, 727, 733, 739, 743, 751, 757, 761, 769, 773, 787, 797, ...

Potências de 2: 256, 512, 1024, 2048, ...

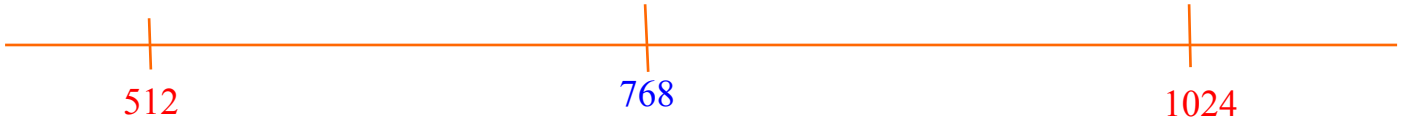
Tabela Hash (Endereçamento Fechado)

Método da Divisão

- Suponha que um conjunto de 2000 elementos será armazenado em uma tabela hash onde o fator de carga estimado é 3. Qual seria a melhor função de hash para o problema usando o método da divisão?

$h(k) = k \% \boldsymbol{m}$

2000/3 ~ 667



Primos: 673, 677,683, 691, 701, 709, 719, 727, 733, 739, 743, 751, 757, 761, 769, 773, 787, 797, ...

Potências de 2: 256, 512, 1024, 2048, ...

Tabela Hash (Endereçamento Fechado - Exemplo)

Suponha as chaves {200, 205, 210, 215, 220, ..., 595}.

- Se eu escolher a função h(k) = k %100. Vai existir colisão em alguma célula? Se sim, quantas?
- Se eu escolher a função h(k) = k %101. Vai existir colisão? Se sim, quantas?

K % 100

0	200, 300, 400, 500
...	
5	205, 305, 405, 505
...	
10	210, 310, 410, 510
...	

K % 101 Multiplos de 101: 101 202 303 404 505

200	205	210	215	220, ... , 295, 300
099	003	008	013	018, ... , 093, 098
305	310	315	320	325, ... , 395, 400
002	007	012	017	022, ... , 092, 097
405	410	415	420	425, ... , 495, 500
001	006	011	016	021, ... , 091, 096

Tabela Hash (Endereçamento Fechado)

Método da Multiplicação

$$f(k) = \lfloor m\,(\underbrace{k.A \% 1}_{\text{Extrai a parte fracionária da k.A}}) \rfloor$$

0 < A < 1

$$A \approx (\sqrt{5} - 1)/2 = 0.6180339887...$$

m não é crítico como no método da divisão

Tabela Hash (Endereçamento Fechado)

Método da Multiplicação (exemplo) $f(k) = \lfloor m\ (k.A\ \%\ 1) \rfloor$

Calcule o código hash das chaves {2, 3, 4, 5, 6, 10, 15}, onde A = 0.2 e m = 1000

Tabela Hash (Endereçamento Fechado)

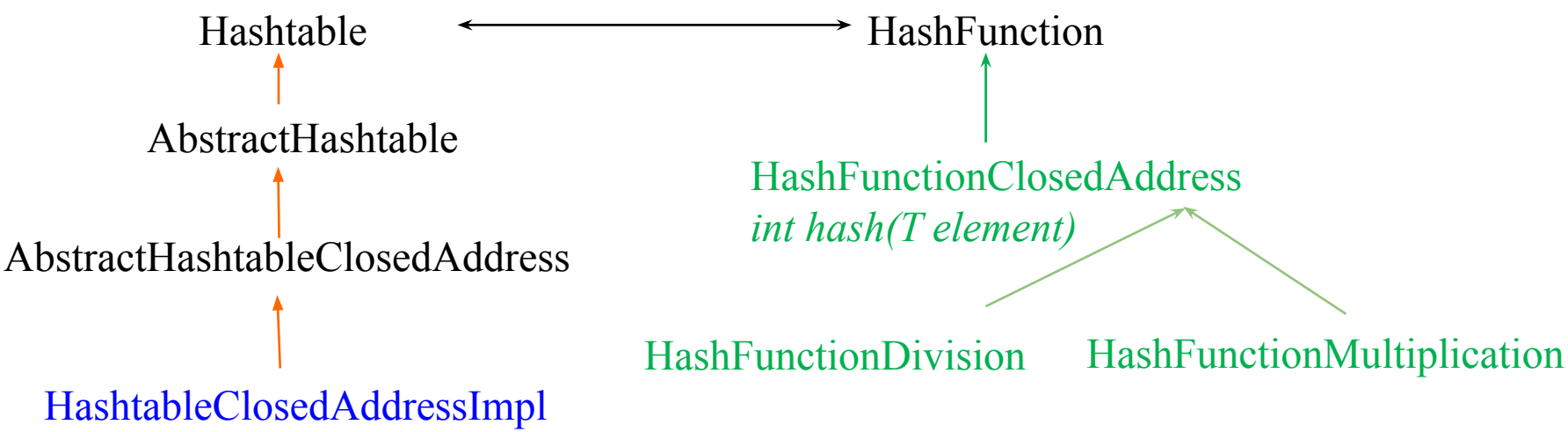
Método da Multiplicação (exemplo) $f(k) = \lfloor m\ (k.A\ \%\ 1) \rfloor$

Calcule o código hash das chaves {2, 3, 4, 5, 6, 10, 15}, onde A = 0.2 e m = 1000

$h(2) = \lfloor 1000\ (2x0.2\ mod\ 1) \rfloor = 400$
$h(3) = \lfloor 1000\ (3x0.2\ mod\ 1) \rfloor = 600$
$h(4) = \lfloor 1000\ (4x0.2\ mod\ 1) \rfloor = 800$
$h(5) = \lfloor 1000\ (5x0.2\ mod\ 1) \rfloor = 000$
$h(6) = \lfloor 1000\ (6x0.2\ mod\ 1) \rfloor = 200$
$h(10) = \lfloor 1000\ (10x0.2\ mod\ 1) \rfloor = 000$
$h(15) = \lfloor 1000\ (15x0.2\ mod\ 1) \rfloor = 000$

Implementação!

Tabela Hash (Endereçamento Fechado)



Implementação!

Tabela Hash (Endereçamento Fechado)

```
public interface Hashtable<T> {

    public boolean isEmpty();
    public boolean isFull();
    public int capacity();
    public int size();
    public void insert(T element);
    public void remove(T element) ;
    public T search(T element);
    public int indexOf(T element);
}
```

```
public interface HashFunction<T> {

}
```

Implementação!

Tabela Hash (Endereçamento Fechado)

```
public abstract class AbstractHashtable<T> implements Hashtable<T> {
    Object[] table;
    int elements;
    int COLLISIONS;
    HashFunction<T> hashFunction;
    public AbstractHashtable(){
      elements = 0;
      COLLISIONS = 0;
    }
    protected void initiateInternalTable(int size){
      this.table = new Object[size];
    }
    public boolean isEmpty() {
      return (elements == 0);
    }
```

Implementação!

Tabela Hash (Endereçamento Fechado)

```
public boolean isFull() {
  return (elements == table.length);
}
public int size() {
  return elements;
}
public int capacity(){
  return this.table.length;
}
public int getCOLLISIONS() {
  return COLLISIONS;
}
```

Implementação!

Tabela Hash (Endereçamento Fechado)

```
public abstract class AbstractHashtableClosedAddress<T>
    extends AbstractHashtable<T> {

    public AbstractHashtableClosedAddress(){
        super();
    }

    @Override
    protected void initiateInternalTable(int size){
        this.table = new LinkedList[size];
}
```

Implementação!

Tabela Hash (Endereçamento Fechado)

```
public interface HashFunction<T> {
}
```

```
public interface HashFunctionClosedAddress<T>
     extends HashFunction<T> {

  public int hash(T element);
}
```

Implementação!

Tabela Hash (Endereçamento Fechado)

```
public class HashFunctionDivisionMethod<T>
     implements HashFunctionClosedAddress<T> {

  protected int tableSize;

  public HashFunctionDivisionMethod(int tableSize) {
    this.tableSize = tableSize;
  }
  public int hash(T element) {
    int hashKey = -1;
    int key = element.hashCode();
    hashKey = (int) key % tableSize;
    return hashKey;
  }
}
```

Implementação!

Tabela Hash (Endereçamento Fechado)

```
public class HashFunctionMultiplicationMethod<T>
     implements HashFunctionClosedAddress<T>{
  protected int tableSize;
  private static final double A = (Math.sqrt(5)-1)/2;
  public HashFunctionMultiplicationMethod(int tableSize){
    this.tableSize = tableSize;
  }
  public int hash(T element) {
    int hashKey = -1;
    int key = element.hashCode();
    double fractionalPart = key*A - Math.floor(key*A);
    hashKey = (int) (tableSize * fractionalPart);;
    return hashKey;
  }
}
```

Implementação!

Tabela Hash (Endereçamento Fechado)

```
public class HashFunctionFactory<T> {
  public static HashFunction createHashFunction(
        HashFunctionClosedAddressMethod method, int tableSize){
        HashFunction result = null;
        switch (method) {
        public enum HashFunctionClosedAddressMethod {
          DIVISION, MULTIPLICATION;                    od(tableSize);
        }
                result = new HashFunctionMultiplicationMethod(tableSize);
        }
        return result;
    }
}
```

Implementação!

Tabela Hash (Endereçamento Fechado)

```
public class HashFunctionFactory<T> {
  public static HashFunction createHashFunction(
        HashFunctionClosedAddressMethod method, int tableSize){
        HashFunction result = null;
        switch (method) {
            case DIVISION:
                result = new HashFunctionDivisionMethod(tableSize);
            case MULTIPLICATION:
                result = new HashFunctionMultiplicationMethod(tableSize);
        }
        return result;
    }
}
```

Implementação!

Tabela Hash (Endereçamento Fechado)

```
public class HashtableClosedAddressImpl<T> extends
    AbstractHashtableClosedAddress<T> {

    public HashtableClosedAddressImpl(int desiredSize,
    HashFunctionClosedAddressMethod method) {
        int realSize  = desiredSize;

        if(method == DIVISION){
          realSize = this.getPrimeAbove(desiredSize);
        }
        initiateInternalTable(realSize);
        HashFunction function =
         HashFunctionFactory.createHashFunction(method,realSize);
        this.hashFunction = function;
    }
```

Implementação!

Tabela Hash (Endereçamento Fechado)

```
    int getPrimeAbove(int number){
      //use Util.isPrime(n)
    }
    public void insert(T element) {
    }
    public void remove(T element) {
    }
    public T search(T element) {
    }
    public int indexOf(T element) {
    }
}
```

Tabelas Hash
Endereçamento Aberto

Tabela Hash (Endereçamento Aberto)

E se o uso de listas para resolver colisões for proibitivo?

Tabela Hash (Endereçamento Aberto)

E se o uso de listas para resolver colisões for proibitivo?

A função de hashing precisa melhorar para buscar uma próxima posição disponível!

Tabela Hash (Endereçamento Aberto)

- Sondagem para buscar um espaço vazio
- Função hash é estendida

$$h : U \times \{0, 1, \ldots, m-1\} \rightarrow \{0, 1, \ldots, m-1\}$$

$$\langle h(k, 0), h(k, 1), \ldots, h(k, m-1) \rangle$$

  - *indice = h(key, probe)*

Tabela Hash (Endereçamento Aberto)

Insirerir a chave k=100

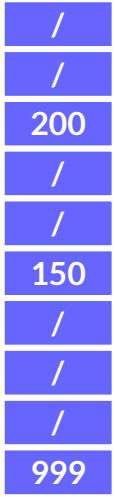


Tabela Hash (Endereçamento Aberto)

Insira a chave k=100

h(100,0)

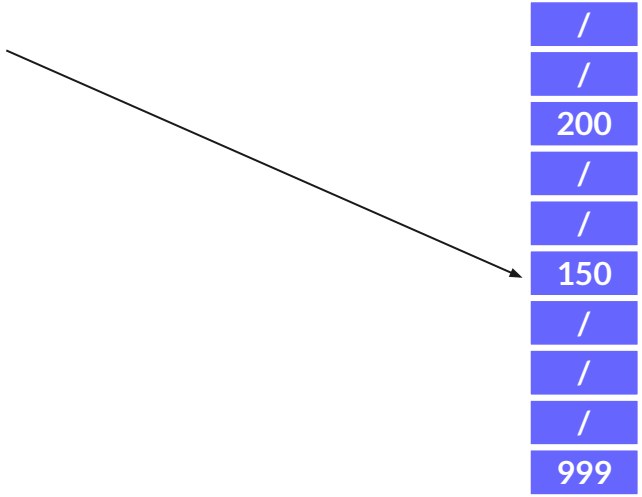


Tabela Hash (Endereçamento Aberto)

Insira a chave k=100

h(100,0)

h(100,1)

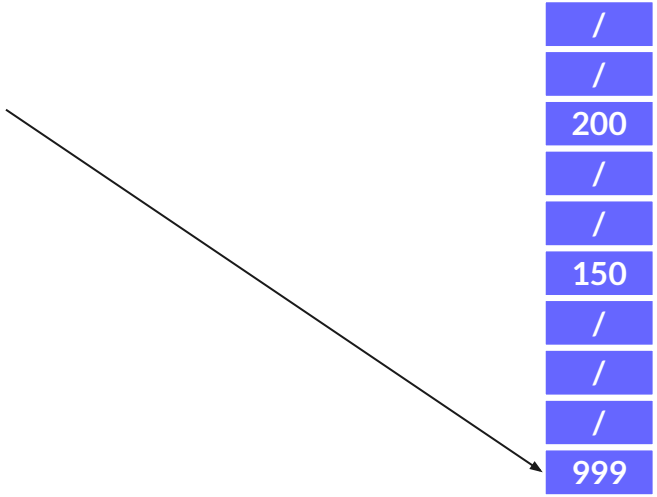


Tabela Hash (Endereçamento Aberto)

Insira a chave k=100

h(100,0)

h(100,1)

h(100,2)

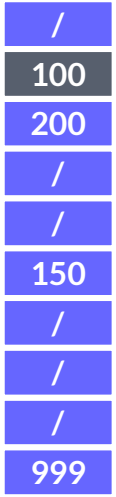


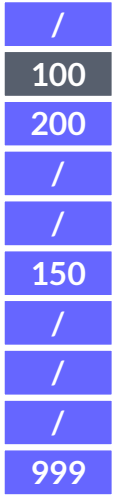
Tabela Hash (Endereçamento Aberto)

Insira a chave k=100

h(100,0)

h(100,1)

h(100,2)




```
HASH-INSERT(T, k)
i ← 0
repeat j ← h(k, i)
        if T[j] = NIL
           then T[j] ← k
                return j
           else i ← i + 1
  until i = m
error "hash table overflow"
```

Tabela Hash (Endereçamento Aberto)

Buscar a chave k=100

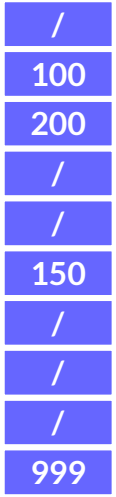


Tabela Hash (Endereçamento Aberto)

Buscar a chave k=100

h(100,0)

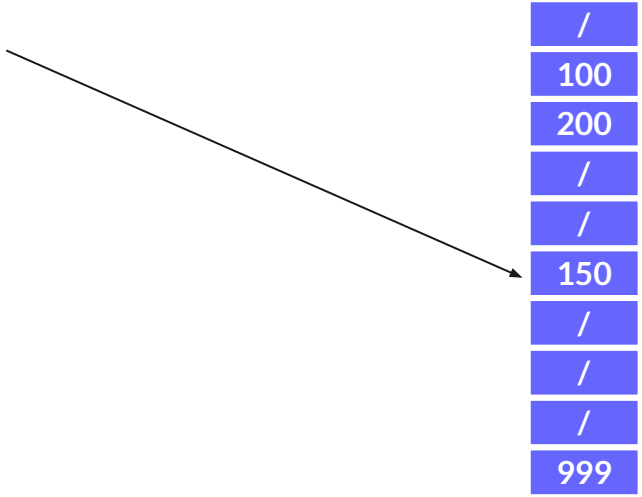


Tabela Hash (Endereçamento Aberto)

Buscar a chave k=100

h(100,0)

h(100,1)

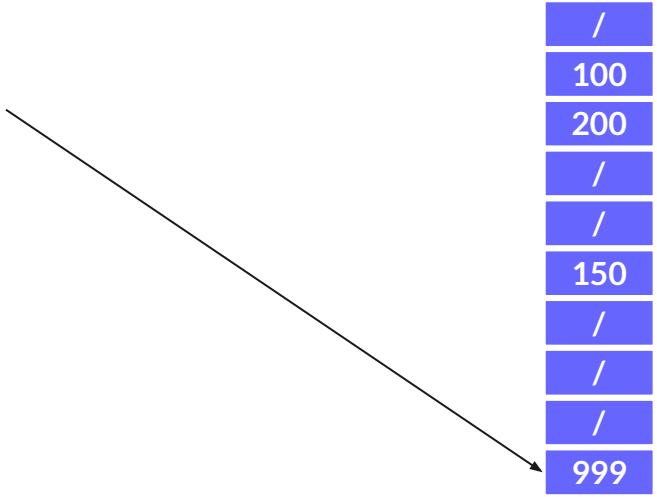
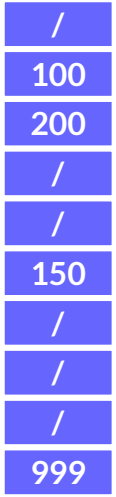


Tabela Hash (Endereçamento Aberto)

Buscar a chave k=100

h(100,0)
h(100,1)
h(100,2)

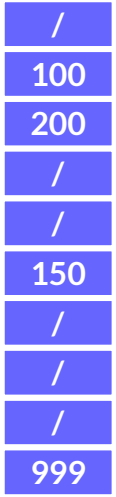



```
HASH-SEARCH(T, k)
i ← 0
repeat j ← h(k, i)
        if T[j] = k
           then return j
        i ← i + 1
  until T[j] = NIL or i = m
return NIL
```

Tabela Hash (Endereçamento Aberto)

Buscar a chave k=100

h(100,0)
h(100,1)
h(100,2)




```
HASH-SEARCH(T, k)
i ← 0
repeat j ← h(k, i)
        if T[j] = k
            then return j
        i ← i + 1
until T[j] = NIL or i = m
return NIL
```

E se a chave não está na tabela?
E se a chave não está na tabela e a tabela está cheia?

Tabela Hash (Endereçamento Aberto)

Remover a chave k=999

/
100
200
/
/
150
/
/
/
999

Tabela Hash (Endereçamento Aberto)

Remover a chave k=999

h(999,0)

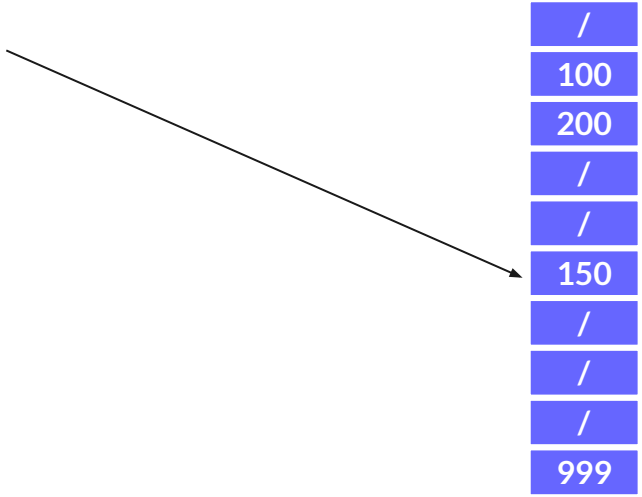


Tabela Hash (Endereçamento Aberto)

Remover a chave k=999

h(999,0)

h(999,1)

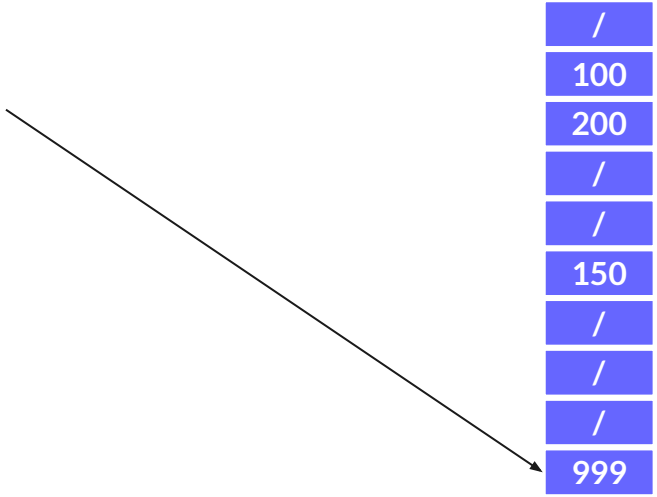
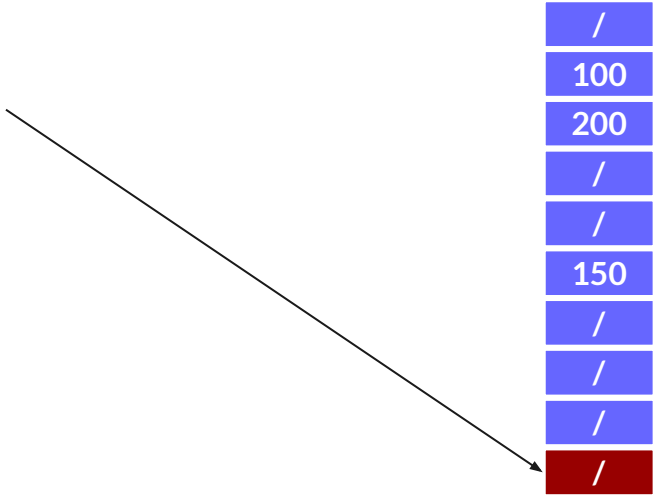


Tabela Hash (Endereçamento Aberto)

Remover a chave k=999

h(999,0)

h(999,1)



O que acontece com a busca pelo 100?

Tabela Hash (Endereçamento Aberto)

Remover a chave k=999

h(999,0)

h(999,1)

```
HASH-INSERT(T, k)
i ← 0
repeat j ← h(k, i)
        if T[j] = NIL || T[j] = DELETED
          then T[j] ← k
               return j
          else i ← i + 1
until i = m
error "hash table overflow"
```

Uso da flag DELETED e ajuste da inserção!

```
HASH-SEARCH(T, k)
i ← 0
repeat j ← h(k, i)
        if T[j] = k
          then return j
        i ← i + 1
until T[j] = NIL or i = m
return NIL
```

Tabela Hash (Endereçamento Aberto)

Quais abordagens de probing?

- Probing linear
- Probing quadrático
- Double hashing

Tabela Hash (Endereçamento Aberto)

Probing linear (simulação)

Tabela Hash (Endereçamento Aberto)

Probing linear

$$h(k,i) = (h'(k) + i) \% m,$$ onde $h'(k)$ é função de hash auxiliar

$h(k,0) = (h'(k) + 0) \% m = h'(k) \% m = (k \% m) \% m = k \% m$
$h(k,1) = (h'(k) + 1) \% m = (h'(k) + 1) \% m = ((k \% m) + 1) \% m = (k \% m) \% 1$
$h(k,2) = (h'(k) + 2) \% m = (h'(k) + 2) \% m = \ldots$
...

Tabela Hash (Endereçamento Aberto)

Probing linear

$$h(k,i) = (h'(k) + i) \% m,$$ onde $h'(k)$ é função de hash auxiliar

$h(k,0) = (h'(k) + 0) \% m = h'(k) \% m = (k \% m) \% m = k \% m$
$h(k,1) = (h'(k) + 1) \% m = (h'(k) + 1) \% m = ((k \% m) + 1) \% m = (k \% m) \% 1$
$h(k,2) = (h'(k) + 2) \% m = (h'(k) + 2) \% m = \ldots$
...

Clustering primário!!!

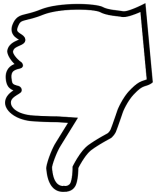


Tabela Hash (Endereçamento Aberto)

Probing quadrático (simulação)

Tabela Hash (Endereçamento Aberto)

Probing quadrático

$$h(k,i) = (h'(k) + c_1.i + c_2.i^2)\% \ m,$$ onde $h'(k)$ é função de hash, c_1, c_2 constantes

$h(k,0) = h'(k) + 0.c_1 + 0.c_2 \% m = h'(k) \% m = (k \% m) \% m = k \% m$
$h(k,1) = (h'(k) + c_1 + c_2) \% m = ((k \% m) + c_1 + c_2) \% m$
$h(k,2) = (h'(k) + 2.c_1 + 4.c_2) \% m = \ldots$
…

Tabela Hash (Endereçamento Aberto)

Probing quadrático

$$h(k,i) = (h'(k) + c_1.i + c_2.i^2) \% m,$$ onde $h'(k)$ é função de hash, c_1, c_2 constantes

$h(k,0) = h'(k) + 0.c_1 + 0.c_2 \% m = h'(k) \% m = (k \% m) \% m = k \% m$
$h(k,1) = (h'(k) + c_1 + c_2) \% m = ((k \% m) + c_1 + c_2) \% m$
$h(k,2) = (h'(k) + 2.c_1 + 4.c_2) \% m = \ldots$
...

Clustering secundário!!!

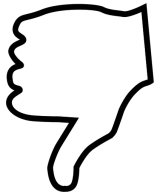


Tabela Hash (Endereçamento Aberto)

Double hashing (simulação)

Tabela Hash (Endereçamento Aberto)

Probing quadrático

$$h(k,i) = (h_1(k) + h_2(k).i) \% m,$$ onde $h_1(k)$ e $h_2(k)$ são funções de hash

$h(k,0) = (h_1(k) + 0.h_2(k)) \% m = h_1(k) \% m$
$h(k,1) = (h_1(k) + h_2(k)) \% m$
$h(k,2) = (h_1(k) + 2.h_2(k)) \% m$
...

Excelentes resultados!
Se aproxima mais do hashing uniforme
Dicas:

- Faça h2(k) retornar ímpar e m uma potência de 2
- Faça m ser um primo e h2(k) retornar um numero menor que m

Tabela Hash (Endereçamento Aberto)

Qual o fator de carga no endereçamento aberto?

E se a tabela encher?

Tabela Hash (Endereçamento Aberto)

Qual o fator de carga no endereçamento aberto?

$$\alpha \leq 1$$

E se a tabela encher?

Tabela Hash (Endereçamento Aberto)

Qual o fator de carga no endereçamento aberto?

$$\alpha \leq 1$$

E se a tabela encher?

Aumentar quando encher
Aumentar quando atingir um limiar (threshold) → Rehash

Tabela Hash (Endereçamento Aberto)

Rehash

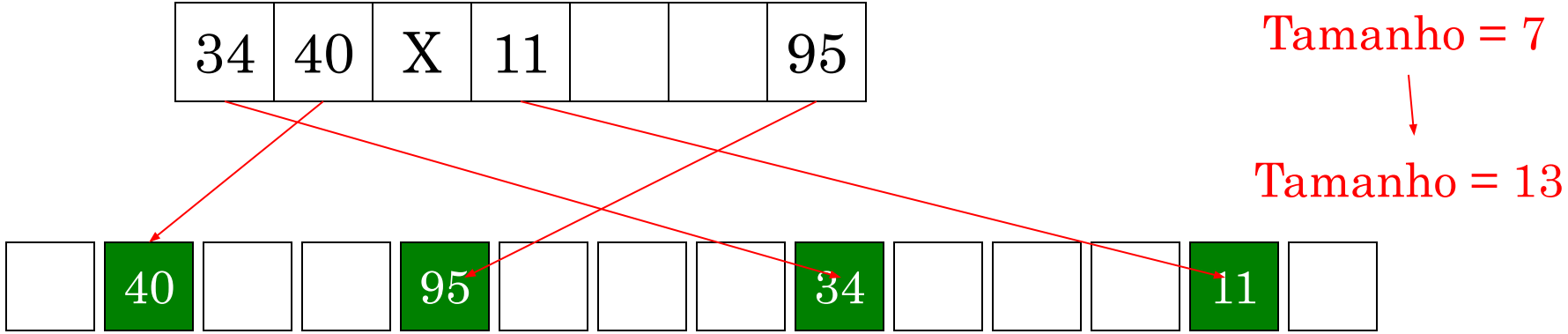


Tabela Hash (Endereçamento Aberto)

Questões pertinentes

- Qual a garantia de que as abordagens percorrem toda a tabela?
  - Probing linear
  - Proging quadrádico
  - Double hashing

Tabela Hash (Endereçamento Aberto)

Questões pertinentes

- Como lidar corretamente com equals e hashCode?
  - hashCode ⇒ equals?
  - equals ⇒ hashCode?

Tabela Hash (Collections)

java.util

Class Dictionary<K,V>

java.lang.Object
    java.util.Dictionary<K,V>

Direct Known Subclasses:

Hashtable

java.util

Class Hashtable<K,V>

java.lang.Object
    java.util.Dictionary<K,V>
        java.util.Hashtable<K,V>

All Implemented Interfaces:

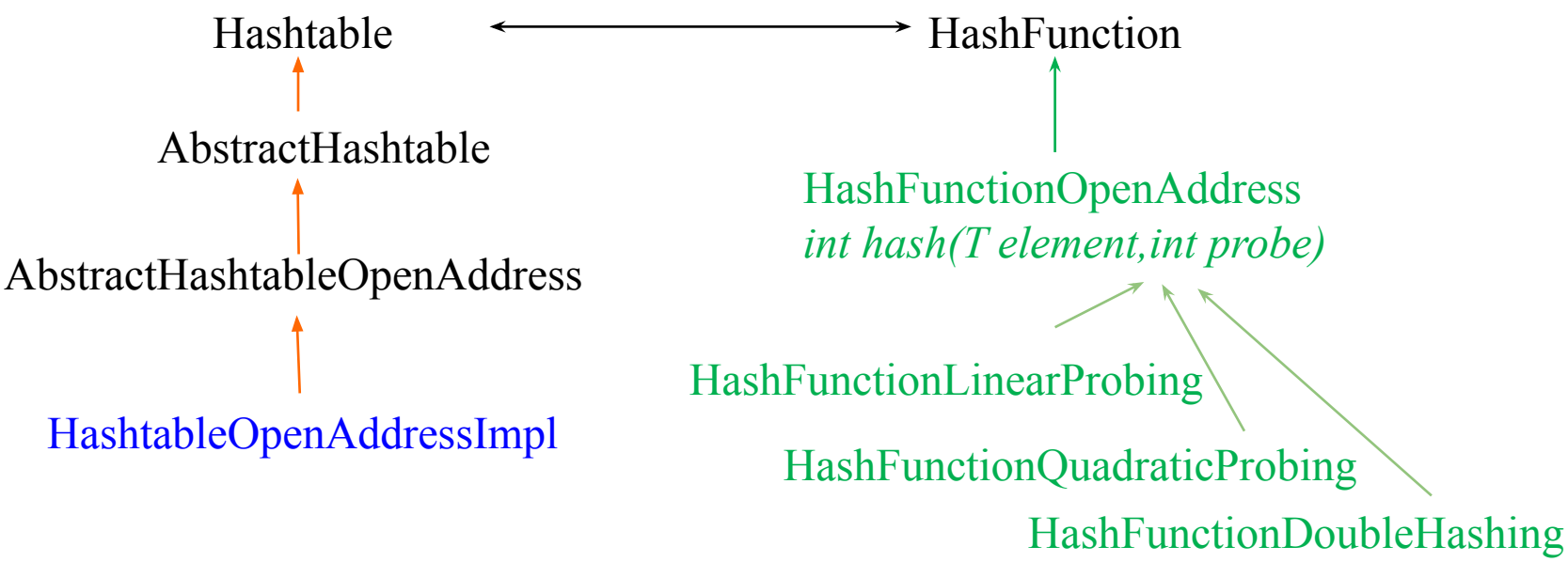
Serializable, Cloneable, Map<K,V>

Direct Known Subclasses:

Properties, UIDefaults

Implementação!

Tabela Hash (Endereçamento Aberto)



Implementação!

Tabela Hash (Endereçamento Aberto)

```
public abstract class AbstractHashtableOpenAddress<T>
     extends AbstractHashtable<T> {

     protected final DELETED deletedElement = new DELETED();
     private int tableSize;

     public AbstractHashtableOpenAddress(){
          this.tableSize = tableSize;
        this.initiateInternalTable(size);
     }
     @Override
     protected void initiateInternalTable(int size){
          this.table = new Storable[size];
     }
}
```

Implementação!

Tabela Hash (Endereçamento Aberto)

```
public abstract class AbstractHashtableOpenAddress<T>
    extends AbstractHashtable<T> {

    protected final DELETED deletedElement = new DELETED();
    private int tableSize;

    public AbstractHashtableOpenAddress(){
        this.tableSize = tableSize;
       this.initiateInternalTable(size);
    }
    @Override
    protected void initiateInternalTable(int size){
        this.table = new Storable[size];
    }
}
```

```
public interface Storable{
}
```

Implementação!

Tabela Hash (Endereçamento Aberto)

```
public interface HashFunction<T> {
}
```

```
public interface HashFunctionOpenAddress<T>
     extends HashFunction<T> {

  public int hash(T element,int probe);
}
```

Implementação!

Tabela Hash (Endereçamento Aberto)

```
public HashFunctionLinearProbing<T>  implements
     HashFunctionOpenAddress<T> {
  protected HashFunctionClosedAddress<T> originalHashFunction;
  protected int tableSize;

  public HashFunctionLinearProbing(int tableSize,
         HashFunctionClosedAddressMethod method) {
     this.tableSize = tableSize;
     if(method == HashFunctionClosedAddressMethod.DIVISION){
         originalHashFunction =  new HashFunctionDivisionMethod<T>(tableSize)
     }else{
         originalHashFunction= new HashFunctionMultiplicationMethod<T> (tableSize);
     }
   }
}
```

Implementação!

Tabela Hash (Endereçamento Aberto)

```
@Override
public int hash(T element, int probe) {
    int generatedIndex = 0;

    generatedIndex=
        ((originalHashFunction.hash(element) + probe) %
    this.tableSize);

    return generatedIndex;
}
```

Implementação!

Tabela Hash (Endereçamento Aberto)

```
public class HashFunctionQuadraticProbing<T> implements     HashFunctionOpenAddress<T> {
 private int tableSize;
 protected HashFunctionClosedAddress<T> originalHashFunction;
 protected int c1, c2;
 public HashFunctionQuadraticProbing(int tableSize,
     HashFunctionClosedAddressMethod method,int c1, int c2) {
   this.tableSize = tableSize;
   if(method == HashFunctionClosedAddressMethod.DIVISION){
     originalHashFunction = new HashFunctionDivisionMethod<T>(tableSize);
   }else{
     originalHashFunction = new HashFunctionMultiplicationMethod<T>(tableSize);
   }
   this.c1 = c1;
   this.c2 = c2;
 }
```

Implementação!

Tabela Hash (Endereçamento Aberto)

```
@Override
public int hash(T element, int probe) {
    int generatedIndex = 0;

    generatedIndex = originalHashFunction.hash(element);
    generatedIndex = ((generatedIndex + c1*probe +      c2*probe*probe)%
    tableSize);

    return generatedIndex;
}
```

Implementação!

Tabela Hash (Endereçamento Aberto)

```
public abstract class AbstractHashtableOpenAddress<T extends Storable>
     extends AbstractHashtable<T> {

     protected final DELETED deletedElement = new DELETED();
     private int tableSize;

     public AbstractHashtableOpenAddress(int size) {
          this.tableSize = size;
        this.initiateInternalTable(size);
     }
     @Override
     protected void initiateInternalTable(int size) {
          this.table = Util.<Storable>makeArray(size);
     }
}
```

Implementação!

Tabela Hash (Endereçamento Aberto)

```
public abstract class AbstractHashtableOpenAddress<T extends Storable>
    extends AbstractHashtable<T> {

    protected final DELETED deletedElement = new DELETED();
    private int tableSize;

    public AbstractHashtableOpenAddress(int size) {
        this.tableSize = size;
       this.initiateInternalTable(size);
    }
    @Override
    protected void initiateInternalTable(int s
        this.table = Util.<Storable>makeArray
    }
}
```

```
public class DELETED
    implements Storable{
    public DELETED() {
    }
}
```

Implementação!

Tabela Hash (Endereçamento Aberto)

```
public class HashtableOpenAddressLinearProbingImpl<T extends Storable>
        extends AbstractHashtableOpenAddress<T> {

    public HashtableOpenAddressLinearProbingImpl(int size,
            HashFunctionClosedAddressMethod method) {
        super(size);
        hashFunction = new HashFunctionLinearProbing<T>(size,method);
        this.initiateInternalTable(size);

    }
    //métodos a implementar
}
```

Implementação!

Tabela Hash (Endereçamento Aberto)

```
public class HashtableOpenAddressQuadraticProbingImpl<T extends Storable>
        extends AbstractHashtableOpenAddress<T> {

    public HashtableOpenAddressQuadraticProbingImpl(int size,
            HashFunctionClosedAddressMethod method,  int c1, int c2) {
        super(size);
        hashFunction = new HashFunctionQuadraticProbing<T>(size,
            method, c1, c2);
        this.initiateInternalTable(size);
    }

   //métodos a implementar
}
```

Implementação!

Tabela Hash (Endereçamento Aberto)

```
public class HashtableOpenAddressDoubleHashing<T extends Storable>
    extends AbstractHashtableOpenAddress<T> {

    public HashtableOpenAddressDoubleHashing(int size,
        HashFunctionClosedAddressMethod methodFunction1,
        HashFunctionClosedAddressMethod methodFunction2) {
        super(size);
        hashFunction = new HashFunctionDoubleHashing<T>(size,
            methodFunction1, methodFunction2);
        this.initiateInternalTable(size);
    }

    //métodos a implementar
}
```